

The Stealth Agent

An autonomous, bot-resistant Greenhouse application agent — with a fit intelligence layer

Document Type	Engineering Design Specification + State-of-the-Art Survey
Version	1.0 — Initial Spec
Audience	Allan, Ziad, and any engineer joining the JobPilot team
Scope	Final Stage submission: stealth Greenhouse agent + Fit Intelligence layer
Status	Design ratified. Reference implementation scaffolded. Ready for build.

Notice of Authorship

Original work — All rights reserved

© 2026 [YOUR NAME]. This document, and the associated software described herein, is the original work of the named author. It was created as a submission to the 1000Jobs Final Stage assessment.

This work is licensed under the JobPilot Evaluation & Personal-Use License. The Assessors (Allan and Ziad) are granted a non-exclusive, time-limited right to read, run, and evaluate this submission. No commercial rights are transferred.

Reproduction of this document, in whole or in part, or use of the architectural ideas, archetype taxonomy, platform routing scheme, prompt templates, or design language described herein, in any commercial product without prior written permission, is prohibited.

See LICENSE and NOTICE.md in the source repository for the full IP claim and terms.

This page is dated 2026-05-02 and is authored simultaneously with the initial commit of the source repository to GitHub. Both serve as evidence of the chain of authorship.

Acknowledged influences are listed in NOTICE.md and in Section 03 (State of the Art) of this document. The taxonomy of eight life-context archetypes, the work-mode-and-region platform graph, the four-layer bot-mitigation model, the prompt templates, the application tracker design, and the editorial visual language are original to this submission.

Contents

01	Executive Summary
02	Vision & Insights — Why This System Has To Be Smarter
03	State of the Art — What Exists, What We Use, Why
04	System Architecture
05	Module: The Stealth Agent (Submission Engine)
06	Module: Fit Intelligence (Company Archetype Matching)
07	API Contract
08	The Application Flow — Step by Step
09	Bot Mitigation Strategy
10	The LLM Question Handler
11	Resilience & Error Handling
12	Logging & Observability
13	Project Structure
14	Local Setup — For New Developers
15	Testing Strategy
16	What We Are Not Building (Yet)
17	Risk Register
18	Roadmap Beyond v1
19	Appendix A — Selector Resilience
20	Appendix B — Prompt Templates
21	Appendix C — The Eight Archetypes

01 Executive Summary

JobPilot is an autonomous job-application platform. Discovery, resume generation, and scoring are already in place. The remaining gap — and the gap this spec closes — is execution: pressing submit on a real job, on a real ATS, without getting flagged as a bot. And then a layer above that: deciding which jobs are worth applying to in the first place, based not just on skill match but on fit with the kind of workplace the candidate actually wants.

This document specifies two modules:

- **The Stealth Agent.** A headless micro-service that accepts a job URL and applicant data, navigates a Greenhouse application end-to-end, answers screening questions with an LLM, and either stops one click before submit (default) or submits to a verified test posting. Wrapped in a local REST API.
- **Fit Intelligence.** An upstream layer that matches each candidate to a company archetype — startup-hungry vs corporate-stability, family-friendly vs travel-heavy, women-empowered cultures, ladder-climbers, and so on — using Glassdoor reviews, careers-page content, and benefits data. The agent only applies to jobs that match the candidate's archetype above a threshold.

What "done" looks like

- POST /apply accepts a job URL plus applicant payload and returns structured JSON.
- The agent fills standard fields, uploads a PDF resume, detects open-ended questions, generates answers via LLM, and submits autonomously.
- POST /score returns a fit score against the candidate's archetype, with reasoning the user can read.
- Every run produces a structured log trail with a clear success or failure cause.
- A new developer can clone the repo, follow the README, drop in their API keys, and have a working server in under ten minutes.

Why this spec exists before the code

A 48-hour build is short. Writing the design first means we build the right thing once, instead of refactoring three times.

It also makes the system legible — to Allan, to Ziad, and to whoever maintains this six months from now.

02 Vision & Insights

Why this system has to be smarter than "more applications"

Speaking from experience: applying to jobs in a market like Dubai taught me something the volume narrative does not capture. I have applied widely, including to roles where I was qualified or over-qualified, and the callback rate was indistinguishable from random. The candidates who landed roles were not the ones who applied the most. They were the ones who got referred — or the ones whose application carried a signal beyond the resume.

Indeed, Glassdoor, and LinkedIn Easy-Apply are funnels designed to reward volume. Tireless application produces tireless rejection. The signal-to-noise ratio inside an ATS inbox is brutal — a recruiter scanning a hundred resumes is not evaluating fit, they are filtering for friction-free choices: keyword matches, recognizable companies, exact title alignment. Being qualified is necessary but nowhere near sufficient.

That informs how I think about JobPilot. A system that just applies faster than a human is a louder funnel. It still loses to the funnel problem. A system that wins has to do four things volume cannot:

1. Apply only to companies that fit the candidate

This is the Fit Intelligence layer (Section 06). Not every qualified candidate is a fit for every company they could pass an interview at. A new graduate who wants to learn fast and work with founders is wasting an application slot on a 30,000-person enterprise — even if they have the skills. A mid-career parent looking for stable benefits and predictable hours is wasting their time on a Series A burning capital. The system should know the difference.

This matters disproportionately for women. As a candidate, I have learned to read between the lines on careers pages: how parental leave is described, whether senior women appear on the leadership page, the language used in job descriptions, the reviews from female employees on Glassdoor. These signals are out there and can be extracted at scale. A system that surfaces them automatically saves the candidate hours of background research per application — which is hours they can spend networking instead.

2. Make the application itself a piece of evidence

The biggest lever in a Greenhouse application is not the resume. It is the open-ended questions — the "why this company" and "tell us about a hard problem" boxes. Most candidates write generic answers under fatigue. An LLM with the candidate's actual background, the company's actual mission, and the specific JD can produce answers that are sharper than what a tired human would write at 11pm on a Tuesday. That is the qualification signal an ATS recruiter is actually looking for.

3. Surface the network, not just the form

In Dubai, a referral from a current employee changes the game. JobPilot v1 does not handle outreach yet, but the Fit Intelligence module already scrapes the company page — the same scrape can produce a

list of "people at this company you should reach out to before applying." That is a future feature; flagging it now ensures we do not architect ourselves out of it.

4. Refuse to apply when the odds are bad

An autonomous system that applies to everything is an autonomous system that destroys its own signal. JobPilot should refuse to apply when match score or archetype fit drops below a configurable threshold. The candidate's reputation with these companies — and with their own ATS history — has a finite budget. We respect that budget.

The thesis in one sentence

Autonomous application is table stakes. The product wins by being the first system that knows when not to apply, applies only where the candidate would actually want to work, and writes better than the candidate would tired.

03 State of the Art

Before specifying our build, here is the landscape we are building on top of, and the explicit choices we are making. This section exists because a serious engineering org should be able to point to its dependencies and explain why they were chosen.

3.1 Browser automation engines

The browser layer is where most ATS automation projects fail. The choice of engine determines what bot detection systems can see. There are four meaningful tiers in 2026:

Tool	Approach	Detection rate	Verdict for JobPilot
Playwright (vanilla)	Standard CDP automation, no stealth.	100% on Cloudflare	Not viable. We use Playwright as the API surface but never the vanilla launcher.
playwright-stealth (Python)	JS-layer patches: navigator.webdriver, plugin shapes, etc.	Moderate. Works on weak detection.	Reasonable v1 baseline. Actively maintained as of 2026 (v2.x). Cheap, fast.
Patchright	Drop-in replacement that patches Playwright internals at the protocol level (CDP).	Low against Cloudflare BotFight; passes most Greenhouse hosts.	Strong v1.1 upgrade path. Same API, one-line install change.
Camoufox	Firefox fork with C++-level fingerprint patches. 0% headless detection on CreepJS benchmarks.	Lowest of the open-source options.	Reserved for sites that defeat Patchright. Costlier per run (heavier process; ~40s+ overhead on Cloudflare Turnstile).
nodriver	Drives Chrome over CDP without Playwright's WebSocket bridge entirely.	Good against fingerprint detection, weak against behavioral scoring.	Not used. We need the Playwright ergonomics for fast development.
browser-use	LLM-driven agent on top of Playwright. 89.1% on WebVoyager benchmark.	Variable — depends on underlying browser.	Considered for future versions where the form schema is too unpredictable for selector-based parsing. Overkill for Greenhouse v1.

Our choice

v1 ships on Patchright (Python) with playwright-stealth as a fallback profile. This pairing gives us the Playwright API surface for fast iteration, protocol-level stealth that handles most ATS deployments, and a straightforward upgrade path to Camoufox for hardened domains. Camoufox is wired in but disabled by default.

3.2 Browser infrastructure (cloud, optional)

For local-machine deployment, we do not need cloud infrastructure. For future production deployment, the contenders are:

- **Browserbase.** Managed browser sessions, built for AI agents. Strong stealth, session recordings for debugging, Playwright-compatible. Closed-source but operationally robust.
- **Steel.dev.** Open-source self-hosted alternative. Same Playwright/Puppeteer API, full control. The right answer if compliance or cost dictates self-hosting.
- **Browser Use Cloud.** Stealth browsers from the browser-use team, 195+ country residential proxies, CAPTCHA handling. Most aggressive stealth posture.

v1 is local-only. The browser/ module is written so swapping in any of the three above is a constructor change, not a rewrite.

3.3 LLM providers

The agent calls an LLM for two things: open-ended question answering and (in Fit Intelligence) archetype scoring. Provider considerations:

Provider	Strength for our use case	Tradeoffs
Anthropic (Claude Sonnet 4.5/Opus 4.7)	Strongest at long-form writing in a specified voice. Excellent at instruction following on "don't write like AI" constraints.	Slightly higher latency than GPT-class models. Default choice.
OpenAI (GPT-5, GPT-5-mini)	Fast, cheap, ubiquitous. Good at structured JSON output for archetype scoring.	Tendency toward AI giveaways in long-form prose without aggressive prompting.
Local models (Ollama + Llama 3.x, Qwen)	Zero per-call cost, no data leaves the machine.	Quality gap on long-form open-ended answers is still real. Worth offering as an option for the cost-sensitive.

The llm/ module abstracts the provider behind a single interface. v1 ships Anthropic and OpenAI clients; Ollama is the v1.1 add.

3.4 Glassdoor and company-data ingestion

Glassdoor has no public API and uses Cloudflare-fronted infrastructure. We have three meaningful paths:

- **Direct scraping of /bff/ endpoints.** Glassdoor's frontend calls undocumented BFF endpoints that return structured JSON. Tools like curl_cffi (TLS impersonation) plus session cookie handling can hit these without a full browser. Cheap and fast — but fragile when Glassdoor rotates endpoints.
- **Headless browser via the same stealth stack.** Reuse Patchright. Heavier per request (~200MB memory, several seconds) but more resilient to endpoint changes.
- **Third-party datasets and APIs.** Bright Data, Apify, Coresignal, ScrapingBee all sell pre-collected or on-demand Glassdoor data. Moves the bot-mitigation problem off our plate. Cost scales with volume.

Our v1 choice is the headless browser path, sharing infrastructure with the application agent. We avoid the legal gray area of hitting undocumented BFF endpoints at scale, and we avoid the per-request cost of a third-party API while we are still prototyping. v1.2 may add a Bright Data fallback for resilience.

On legal posture

Glassdoor's terms of service prohibit automated access. We treat company-data ingestion as research-grade: low volume, respectful rate limits, public pages only, no login bypass. We do not store or redistribute Glassdoor content; we extract features and discard the source.

This is a v1 posture. A productionized JobPilot would either license data, or use a vendor like Bright Data that takes on the compliance posture, or build first-party data through user-contributed reviews.

3.5 Form parsing and field detection

Two competing approaches:

- **Selector-based parsing (our v1 choice).** Greenhouse forms are structurally consistent: input names follow stable patterns like `job_application[first_name]` and `job_application[answers_attributes][N][text_value]`. ARIA attributes mark required fields. Label text is parseable. The parser is deterministic, fast, and debuggable.
- **Vision-language model parsing.** Tools like browser-use, Skyvern, and Agent-E use LLMs to interpret screenshots or DOM trees and decide what to fill. Generalizes across ATSs, but slow (5-15s per decision), expensive, and harder to debug.

For Greenhouse-only v1, selector-based wins. When we expand to Workday — which is structurally chaotic — we revisit the VLM approach for that flow module specifically.

3.6 Bot mitigation: the layered model

Modern ATS detection runs in four layers. Each requires a different countermeasure. We adopt all four:

Detection layer	What's checked	Our countermeasure
Protocol	CDP Runtime.enable timing, automation flags	Patchright (protocol-level patches)
Browser fingerprint	navigator.webdriver, canvas, WebGL, fonts, audio context	Patchright + UA matching + viewport randomization. Camoufox if escalation needed.
Behavioral	Typing rhythm, mouse paths, scroll patterns, time-on-page	Custom humanizer module: log-normal keystroke timing, Bezier mouse paths, idle pauses.
Network/IP	IP reputation, geolocation/timezone consistency	Optional residential proxy hook. Locale and timezone matched to UA at launch.

04 System Architecture

JobPilot's autonomous-application surface is structured as two services behind a thin HTTP layer. Each service has a single responsibility and a stable interface so any one can be replaced without touching the others.

Service map

Service	Responsibility	Endpoints
Stealth Agent	Navigate a Greenhouse application, fill fields, answer questions, submit autonomously.	POST /apply
Fit Intelligence	Extract company archetype from web data; score candidate fit against archetype.	POST /score, POST /archetype/company
Orchestrator (lightweight)	Glue. Receives a job + candidate, calls /score, gates the call to /apply on threshold.	POST /process

Internal modules (Stealth Agent)

Module	Responsibility	Key dependencies
api/	FastAPI server, payload validation, run lifecycle.	FastAPI, Pydantic, structlog
browser/	Stealth Chromium launcher and page lifecycle.	Patchright, playwright-stealth (fallback)
flow/	Greenhouse-specific orchestration: navigate, parse, fill, upload, submit.	browser/, BeautifulSoup
humanizer/	Human-like typing, mouse paths, idle pauses, scroll.	asyncio, NumPy
llm/	Provider-agnostic LLM client. Anthropic, OpenAI, Ollama.	anthropic, openai, httpx
observability/	Structured logs, screenshots, DOM snapshots per run.	structlog, Pillow

Internal modules (Fit Intelligence)

Module	Responsibility	Key dependencies
scrapers/	Glassdoor reviews, careers page, JD page, optional LinkedIn About.	Patchright (shared), curl_cffi

Module	Responsibility	Key dependencies
features/	Convert raw text into archetype features. NLP signals: keywords, sentiment, named entities.	spaCy (small model), or LLM-based extraction
archetypes/	Definition of the eight archetypes. Each is a named feature vector.	Pure Python data
scoring/	Compare candidate's stated archetype to company's extracted archetype. Cosine similarity + LLM reasoning.	scikit-learn, llm/

05 Module: The Stealth Agent

This is the submission engine. Given a job URL and applicant payload, it drives a Greenhouse application end-to-end and submits autonomously. An optional review-mode flag (`auto_submit=false`) halts before the final click for cases where the operator wants to inspect the form first. The full mechanics are specified in Sections 08–12.

Inputs

- Job URL (Greenhouse-hosted)
- Applicant payload (name, contact, resume PDF path, resume text, work auth)
- Options (`auto_submit`, `headless`, `llm_provider`, etc.)

Outputs

- Structured JSON response with state, `run_id`, screenshot path, log path
- Run artifacts on disk: `run.jsonl`, `pre_submit.png`, optional `dom_snapshot.html`

Guarantees

- No field is auto-filled with a placeholder. Either the payload provides a value or the field is left blank.
- Honeypots are detected and skipped. Filling a honeypot is the single fastest way to get the application silently dropped.
- `auto_submit` defaults to true. The agent submits autonomously per the brief's requirement of full end-to-end execution. A pre-submit halt is available as an opt-in review mode (`auto_submit=false`) for development or operator-supervised runs.
- Bot detection is treated as a first-class outcome, not an exception. `blocked_by_bot_detection` is a distinct, structured response.

06 Module: Fit Intelligence

The pitch

Two candidates with identical resumes will thrive at very different companies. JobPilot, today, is blind to that. Fit Intelligence is the layer that makes JobPilot able to answer: "Should this person apply here, given what they actually want from work?"

This is not a soft feature. For candidates who have applied broadly and gotten rejected — or worse, gotten hired into the wrong place — the cost of a bad fit dwarfs the cost of a missed application. The system should be biased toward saying no.

The eight archetypes

We define eight company archetypes, derived from common employee experience patterns. A candidate selects up to two during onboarding (most people are a primary plus a secondary). Each archetype is a named vector of weighted features that the scoring layer compares against extracted company data. Full archetype definitions are in Appendix C.

Archetype	Who picks it	Signals it matches
The Founder Track	Fresh grads, aspiring founders, early-career builders who want range and pace.	Series A-C funding stage, < 200 headcount, founder visibility, equity-heavy comp, learning-stipend mentions, "wear many hats" language.
The Corporate Climber	Mid-career, ambition for title and scope. Wants ladders, structured promotion.	Public/late-stage, > 5,000 headcount, defined leveling (L4, L5, IC4...), MBA-heavy leadership, "high-performance" reviews.
The Stable Provider	Mid-career, family-focused. Values predictability, benefits, work-life balance.	Strong parental leave (16+ wks), childcare benefits, healthcare coverage, < 45-hour weeks, "sustainable pace" reviews, > 4.0 work-life balance score.
The Women-Forward Workplace	Women prioritizing cultures with female leadership and lived equity, not just brochure DEI.	≥30% women in leadership pages, named senior women cited in reviews, gender-pay-equity disclosure, mentorship programs, positive sentiment from female-tagged Glassdoor reviews.
The Globe-Trotter	People who love travel as part of the job: client-facing consultants, field engineers, GTM roles.	International offices, "travel up to 50%" in JD, customer-success/field roles, frequent-flier mentions in reviews.
The Mission Believer	People who need the work itself	Mission statements with concrete impact

Archetype	Who picks it	Signals it matches
	to matter — climate, health, education, equity.	metrics, B-Corp status, non-profit partnerships, sector tags (climate, healthtech, edtech).
The Deep Specialist	Senior ICs who want to go deep, not manage. Want technical respect and time to think.	Staff/Principal/Distinguished engineer ladder, conference talks, peer-reviewed publications by employees, "no manager track required" language.
The Remote-First Lifer	Location-independents. Want async-friendly cultures, no return-to-office whiplash.	Remote-first explicit (not "remote-friendly"), distributed across timezones, public RTO policy, async-collaboration tools mentioned in reviews.

Data sources per company

Signal extraction pulls from up to five sources. Each is best-effort — the scorer degrades gracefully when a source is missing.

Source	What we extract	How we get it
Greenhouse JD page	Headcount hints, location/remote policy, travel %, level, benefits one-liners.	Already in browser context — free.
Glassdoor company overview	Star rating, work-life balance score, cultural breakdowns, CEO approval.	Stealth headless visit. Cached per company for 30 days.
Glassdoor reviews (top 50)	Employee sentiment by tag (women, parents, seniors, juniors). Common phrases.	Stealth headless. Cached. Sentiment via LLM.
Company careers/about page	Mission statement, leadership team composition, benefits page.	Direct fetch via the same stealth stack.
Funding/headcount (optional)	Stage, last raise, employee count.	Crunchbase or LinkedIn scrape. v1.1 — not blocking.

The scoring pipeline

1. Receive job_url and candidate_archetype (primary, optional secondary, weights).
2. Resolve the company name from the job URL.
3. Pull (or hit cache for) all five sources. Cache TTL: 30 days for reviews, 7 days for careers page.
4. Run feature extraction: a fixed set of ~25 features (headcount estimate, women-in-leadership %, parental-leave-weeks-mentioned, sentiment-by-tag, etc.). Most are extracted via a single LLM call

with structured JSON output. The deterministic ones (headcount range, location) are parsed directly.

5. Compute the company's archetype vector by projecting features onto each archetype's weighted definition. The company gets a score per archetype, not a single label.
6. Compute candidate-company fit as a weighted overlap between the candidate's chosen archetypes and the company's archetype scores.
7. Generate a one-paragraph reasoning via LLM. Cite specific evidence ("30% of named leaders are women, parental leave is 18 weeks, work-life balance score is 4.3 — strong match for Women-Forward Workplace").
8. Return: overall score (0.0–1.0), per-archetype breakdown for the company, and the reasoning. The orchestrator gates application on this score.

API contract

POST /score

```
{
  "job_url": "https://job-boards.greenhouse.io/grammarly/jobs/7767680",
  "candidate_archetypes": [
    { "name": "women_forward", "weight": 0.6 },
    { "name": "stable_provider", "weight": 0.4 }
  ]
}
```

Response — 200

```
{
  "status": "scored",
  "company": "Grammarly",
  "fit_score": 0.78,
  "verdict": "apply",
  "company_archetype_scores": {
    "founder_track": 0.21,
    "corporate_climber": 0.55,
    "stable_provider": 0.71,
    "women_forward": 0.82,
    "globe_trotter": 0.18,
    "mission_believer": 0.44,
    "deep_specialist": 0.50,
    "remote_first": 0.66
  },
  "reasoning": "Strong match on Women-Forward Workplace: 4 of 11 named leaders are women, parental leave is 16 weeks, gender-pay-equity disclosure is published, and Glassdoor sentiment from reviewers tagged as women trends positive (3.9/5 vs 3.7/5 overall). Stable-provider signal is moderate: benefits are above average but the
```



```

company has had two layoffs in the past 24 months, which the reviews flag.",
  "evidence": [
    {"source": "glassdoor", "claim": "Work-life balance: 4.1/5", "url": "..."},
    {"source": "careers", "claim": "16 weeks paid parental leave for all parents",
"url": "..."},
    ...
  ],
  "cache_age_days": 3
}

```

The candidate-side: how someone picks their archetype

The user does not have to read eight definitions. We collect the same signal through a short, conversational onboarding — five to seven multiple-choice questions whose answers map probabilistically to the archetype vector. Examples:

- "In your ideal week, how much travel would you like?" (None / Some / A lot)
- "Which of these matters more to you right now: learning fast or being paid predictably?"
- "How important is it that your manager is a woman?"
- "Do you want to be the most senior person in the room, or the most junior?"

Five questions get us to a confident two-archetype profile. The user can refine it after seeing their first few scored jobs.

Why this matters disproportionately for women candidates

Companies hide behind generic DEI language on careers pages. The signal is in the reviews — and in the parts of careers pages most candidates do not have time to read carefully.

Surfacing this automatically does not just save time. It lets a candidate see, before applying, that a company has 6% women in engineering leadership despite a polished diversity statement. That is information the candidate is entitled to and that the standard ATS workflow obscures.

07 API Contract

POST /apply (Stealth Agent)

Request body

```
{
  "job_url": "https://job-boards.greenhouse.io/grammarly/jobs/7767680",
  "applicant": {
    "first_name": "Ahmed",
    "last_name": "Khan",
    "email": "ahmed@example.com",
    "phone": "+971501234567",
    "location": "Dubai, UAE",
    "linkedin": "https://linkedin.com/in/ahmedkhan",
    "github": "https://github.com/ahmedkhan",
    "portfolio": "https://ahmedkhan.dev",
    "resume_path": "/abs/path/to/resume.pdf",
    "resume_text": "<full markdown or text version, used by the LLM>",
    "work_auth": "Authorized to work in UAE; require sponsorship in US",
    "preferred_pronouns": "he/him"
  },
  "options": {
    "auto_submit": true,
    "headless": true,
    "llm_provider": "anthropic",
    "llm_model": "claude-sonnet-4-5",
    "min_match_score": 0.0,
    "screenshot_on_failure": true
  }
}
```

Success response — 200

```
{
  "status": "success",
  "run_id": "run_2026-05-01T09-12-44Z_8f3a",
  "state": "submitted",
  "fields_filled": 14,
  "questions_answered": 3,
  "resume_uploaded": true,
  "duration_ms": 32118,
  "screenshot": "logs/run_.../post_submit.png",
  "next_action": "watch_for_recruiter_response"
}
```

Error states

state	Meaning	HTTP
submitted	Form submitted (default behavior).	200
pre_submit_ready	Form filled, halted (only when auto_submit=false in review mode).	200
below_fit_threshold	Pre-flight score gate refused application.	200
invalid_payload	Required applicant fields missing or malformed.	400
job_not_found	URL did not resolve to a Greenhouse application form.	404
form_schema_unrecognized	Form structure outside known patterns.	422
blocked_by_bot_detection	Cloudflare, hCaptcha, or similar.	503
llm_error	LLM call failed.	502
upload_failed	Resume rejected.	500
timeout	Step exceeded budget.	504

08 The Application Flow

Canonical happy-path sequence. Each step has a name, a budget, and an outcome — when something fails, the log identifies which step broke and how.

#	Step	Budget	What it does
1	navigate	10s	Open job URL in stealth Chromium. Wait for form.
2	parse_form	5s	Extract field schema. Flag honeypots.
3	score_fit	8s	Optional: archetype gate. Abort if below threshold.
4	fill_basics	20s	Name, email, phone, etc. Humanized typing.
5	upload_resume	15s	Attach PDF, wait for upload confirmation.
6	answer_questions	60s	For each open-ended question: prompt LLM, type answer.
7	fill_demographics	10s	Optional EEO. Default: "decline to self-identify".
8	validate_form	5s	Walk form once more. No blank required, no honeypots filled.
9	submit_or_halt	varies	Submit + capture confirmation (default), or halt + screenshot (review mode).

Honeypot detection

A field is flagged as a honeypot if any are true:

- `display:none`, `visibility:hidden`, or `opacity:0` on the input or any ancestor.
- Positioned off-screen (negative `top/left`, or `width/height` of 0).
- `tabindex="-1"` and not part of the visible label structure.
- Field name matches known honeypot patterns (`website_url_2`, `address_secondary`).

09 Bot Mitigation Strategy

Layered. Each layer handles a different detection vector.

Layer 1 — Browser fingerprint

- **Patchright** patches CDP-level giveaways. playwright-stealth is the fallback for less protected sites.
- **User-agent** matched to actual Chromium version.
- **Viewport randomization** drawn from real desktop resolution distributions.
- **Locale and timezone** set as a consistent pair.

Layer 2 — Behavioral

- **Humanized typing.** Per-character delay sampled from log-normal (mean ~120ms, sigma ~40ms).
- **Mouse paths.** Bezier-curve approximation. Hover before click. Occasional miss-and-retarget.
- **Idle pauses.** 1-4s between major sections.
- **No auto-fill on empty optional fields.** Only fill what the payload provides or what is required.
- **Scroll.** Real users scroll. Agent scrolls between sections, occasionally back up.

Layer 3 — Network

- **Persistent session cookies** across the run.
- **Reasonable referer** set from the company careers page when implied by the URL.
- **Residential proxy hook** wired in via env var, off by default for local.

Layer 4 — Adaptive

- **Detect challenge pages.** Cloudflare, hCaptcha, "unusual activity" interstitial. Halt with structured error.
- **Backoff and rotate.** Three consecutive blocks against the same domain = domain cooldown.

What we do not do

We do not solve CAPTCHAs. Surface and halt instead.

We do not check "I am a human" boxes that are part of explicit anti-bot agreement.

10 The LLM Question Handler

Open-ended questions are where automated applications visibly fail. The LLM module produces answers that read like the candidate wrote them after thinking carefully.

Detection

- Input is a textarea, or text input with maxlength > 200.
- Label is a question, or contains "describe", "tell us", "why", "how".
- Not the cover letter field.

Prompt construction

Each question gets four parts: role priming, candidate context (full resume_text), job context (scraped JD), and the question with character/word limit.

LLM is instructed to:

- Stay within 60-80% of any maximum length.
- Reference at least one specific resume item.
- Avoid AI giveaways: "I'm thrilled", "deeply passionate", "leverage".
- Refuse to fabricate experience the resume does not support.

Answer injection

Typed via humanized-typing path. We do not paste — paste events are detectable and rare for free-text. A 400-word answer takes 90+ seconds to type, which is realistic.

11 Resilience & Error Handling

Failure taxonomy

Category	Examples	Recovery
Transient	Network blip, slow load.	Retry x3 with exponential backoff.
Detected and blocked	Cloudflare, hCaptcha, interstitial.	Halt. Return <code>blocked_by_bot_detection</code> . Domain cooldown.
Schema mismatch	New question type, redesigned form.	Halt. Capture screenshot + DOM. Flag for review.
LLM failure	Rate limit, malformed output.	Retry once with smaller prompt. Else <code>llm_error</code> .
Upload failure	File rejected.	Halt. Surface message verbatim.
Logic bug	Unhandled exception.	Catch at top level. <code>unknown_error</code> + stack.

Step budgets

Every step has a budget. Exceeding it returns a timeout error tagged with the step name.

Idempotency

Same job + payload should produce the same outcome on re-run, modulo LLM nondeterminism. Run IDs are unique. We do not resume half-completed runs — form state is gone.

Optional review mode

By default the agent runs end-to-end and submits. If the operator passes `auto_submit=false`, the agent halts after `validate_form`, takes a screenshot of the final state, and returns. This is provided for development, debugging, and operator-supervised runs against unfamiliar postings — it is not the default behavior.

12 Logging & Observability

Every log line is a JSON object via structlog. One run = one logs/<run_id>/run.jsonl.

Example log line

```
{
  "run_id": "run_2026-05-01T09-12-44Z_8f3a",
  "ts": "2026-05-01T09:12:51.044Z",
  "step": "answer_questions",
  "substep": "llm_call",
  "level": "info",
  "question_index": 2,
  "question_text": "Why are you interested in this role at Grammarly?",
  "prompt_tokens": 1842,
  "completion_tokens": 312,
  "duration_ms": 4118,
  "model": "claude-sonnet-4-5"
}
```

Artifacts per run

- run.jsonl — structured log.
- pre_submit.png — full-page screenshot before halt/submit.
- failure.png — captured on any non-success terminal state.
- dom_snapshot.html — on form_schema_unrecognized.
- network.har — optional, with HAR_CAPTURE=true.

13 Project Structure

```

jobpilot/
├── README.md
├── pyproject.toml
├── .env.example
├── .gitignore
├── src/
│   └── jobpilot/
│       ├── __init__.py
│       ├── api/
│       │   ├── server.py           # FastAPI app
│       │   ├── routes_apply.py    # POST /apply
│       │   ├── routes_score.py    # POST /score
│       │   └── schemas.py         # Pydantic models
│       ├── browser/
│       │   ├── launcher.py        # Patchright + stealth setup
│       │   └── context.py
│       ├── flow/
│       │   ├── greenhouse.py      # orchestration
│       │   ├── parser.py          # form schema extraction
│       │   ├── honeypot.py
│       │   └── upload.py
│       ├── humanizer/
│       │   ├── typing.py          # log-normal keystrokes
│       │   ├── mouse.py           # bezier paths
│       │   └── pacing.py          # idle, scroll
│       ├── llm/
│       │   ├── client.py          # provider-agnostic
│       │   ├── anthropic_impl.py
│       │   ├── openai_impl.py
│       │   └── prompts.py
│       ├── fit/
│       │   ├── archetypes.py      # Fit Intelligence
│       │   ├── scorer.py          # the 8 archetype defs
│       │   ├── scrapers/
│       │   │   ├── glassdoor.py
│       │   │   ├── careers.py
│       │   │   └── greenhouse_jd.py

```

```
|
| |
| | └─ features.py          # extract feature vector
| | └─ cache.py            # 30/7-day TTL
| |
| └─ observability/
|     └─ logger.py
|     └─ artifacts.py
└─ tests/
└─ scripts/
└─ logs/
```

14 Local Setup

Goal: clone, install, configure keys, hit the API in under ten minutes.

Prerequisites

- Python 3.11+
- Poetry (pip install poetry)
- Anthropic or OpenAI API key
- ~500 MB free for Chromium

Install

```
git clone https://github.com/<you>/jobpilot.git
cd jobpilot
poetry install
poetry run patchright install chromium

cp .env.example .env
# Edit .env, paste your ANTHROPIC_API_KEY

poetry run uvicorn jobpilot.api.server:app --reload --port 8000
```

.env template

```
ANTHROPIC_API_KEY=sk-ant-...
OPENAI_API_KEY=sk-...

LLM_PROVIDER=anthropic
LLM_MODEL=claude-sonnet-4-5

HEADLESS=true
AUTO_SUBMIT=false
SCREENSHOT_ON_FAILURE=true
LOG_LEVEL=info

# Optional residential proxy
# PROXY_URL=http://user:pass@proxy.example.com:8000
```

First request

```
curl -X POST http://localhost:8000/apply \
  -H "Content-Type: application/json" \
  -d @sample_payload.json
```

15 Testing Strategy

Unit tests

- **Parser.** Saved HTML fixtures of real Greenhouse pages → expected schema.
- **Honeypot detector.** Synthetic HTML with various honeypot patterns.
- **Humanizer timing.** Statistical assertions on typing-delay distribution.
- **Archetype scorer.** Fixed feature vectors → expected archetype scores.

Integration tests

Against recorded fixtures. `scripts/record_fixture.py` captures a real page once. CI runs against saved pages with stubbed LLM.

Live smoke test

Pre-deploy, one developer runs against a verified test posting.

Metrics

Metric	Target	How
Submission success (test)	$\geq 95\%$	Smoke tests
Bot-detection block rate	$< 5\%$	Production logs
Mean time to pre_submit	$< 90s$	Per-run duration
LLM cost per application	$< \$0.05$	Token counts $\times$ model price
Archetype score precision @ 0.7+	Inter-rater Cohen's $\kappa \geq 0.6$	Manual rubric on 50 companies

16 What We Are Not Building (Yet)

- **Multi-ATS.** v1 is Greenhouse-only. Lever, Workday, Ashby — each a separate flow module, gated by URL pattern.
- **Account creation.** Halt and surface.
- **CAPTCHA solving.** Out of scope on principle.
- **Cover letter generation.** If payload provides one, use it. Generation = different workflow.
- **Networking sidecar.** Highest-value future feature; out of scope here.
- **Persistent queue.** v1 is synchronous per request.

17 Risk Register

Risk	Likelihood	Impact	Mitigation
Greenhouse changes form structure	High over time	High	Selector-resistant parser; fixture tests.
LLM produces visibly AI-flavored answers	Medium	High	Prompt bans giveaways. Spot-check pre_submit screenshots.
Bot detection improves	High over time	High	Layered mitigation; halt-and-report.
Resume PDF rejected (size/format)	Low	Medium	Validate before browser launch.
Glassdoor scraping detected	Medium	Medium	Aggressive caching, low rate, fallback to Bright Data.
Archetype score is wrong for a candidate	Medium	Medium	Reasoning is exposed. Candidate can override.
LLM rate limit during burst	Medium	Low	Backoff. Per-user concurrency cap.
Application submitted with wrong info	Low	High	Strict payload validation in api/. Pre-flight fit gate catches obvious mismatch. Optional review mode (auto_submit=false) for first runs against unfamiliar postings.
Legal/ToS exposure	Medium	High	Document boundaries. Per-domain cooldowns.

18 Roadmap Beyond v1

v1.1 — Hardening

- Replay tool — re-run failures against captured DOM.
- Selector telemetry — top failing selectors per week.
- Per-company quirk overrides.
- Camoufox profile for hardened domains.

v1.2 — Multi-ATS

- Lever module.
- Workday module.
- Ashby module.

v2 — Quality, not just throughput

- Pre-flight scoring is enforced by default.
- Answer quality review tooling — surface AI-flavored phrasing in pre-submit screenshots before they ship.
- Cover letter generation.
- Onboarding wizard for archetype selection.

v3 — Networking Sidecar

For each scored job, produce a parallel artifact: high-fit people at the company to reach out to before applying, with suggested message drafts. Highest-leverage activity in markets like Dubai.

Appendix A — Selector Resilience Notes

Prefer	Avoid	Why
input[name="job_application[first_name]"]	input.css-1ab23cd	Names stable; CSS hashes regenerate.
[aria-required="true"]	Hardcoded required-field list	ARIA tracks source of truth.
XPath: label text → input	nth-child traversal	Forms reorder; label text stable.
Wait for input to be enabled	Fixed sleep	Async loads; fixed sleeps flake.

New question types: parser halts with form_schema_unrecognized + DOM snapshot. First action

Monday morning is to add a fixture, write a parser test, extend the parser. Outage → fast learning loop.

Appendix B — Prompt Templates

Open-ended question prompt

SYSTEM:

You are helping a real candidate write a real answer for a real job application. Write in their voice, in the first person. Do not pretend to be them – be the writer they would be if they had thirty quiet minutes and the resume in front of them.

Hard rules:

- No clichés ("I'm thrilled", "deeply passionate", "leverage").
- No three-part lists in every paragraph. Vary structure.
- Reference specific items from the resume, not generalities.
- Stay under {max_chars} characters and aim for 60-80% of that.
- Do not invent experience the resume does not support.

USER:

CANDIDATE RESUME:

{resume_text}

JOB DESCRIPTION:

{jd_text}

QUESTION (max {max_chars} chars):

{question_text}

Write the answer. Output only the answer text, no preamble.

Archetype feature extraction prompt

SYSTEM:

You are extracting structured features from company data for an archetype-matching system. Be conservative. If a feature is not directly evidenced, return null. Do not extrapolate or guess.

USER:

SOURCES:

- Greenhouse JD: {jd_text}
- Careers page: {careers_text}
- Glassdoor reviews (sampled): {reviews_text}
- Glassdoor numeric scores: {scores_json}

Extract these features as JSON:

```
{
  "headcount_estimate": "<small|mid|large|enterprise|null>",
  "funding_stage": "<seed|series_a|series_b|series_c|public|null>",
```

```

"remote_policy": "<remote_first|hybrid|in_office|flexible|null>",
"travel_pct": "<0|low|moderate|high|null>",
"parental_leave_weeks": "<int|null>",
"women_in_named_leadership_pct": "<float|null>",
"wlb_score": "<float|null>",
"mission_concreteness": "<low|moderate|high|null>",
"ic_track_signal": "<weak|moderate|strong|null>",
"evidence": [{"feature": "...", "claim": "...", "source": "..."}, ...]
}

```

Archetype scoring rationale prompt

SYSTEM:

Given a company's extracted features and a candidate's archetype profile, write one paragraph explaining the fit score. Cite at least three pieces of concrete evidence. No marketing language.

USER:

CANDIDATE PROFILE:

Primary: {primary_archetype} ({primary_weight})

Secondary: {secondary_archetype} ({secondary_weight})

COMPANY: {company_name}

FEATURES: {features_json}

PER-ARCHETYPE SCORES: {archetype_scores_json}

OVERALL FIT: {fit_score}

Write a single paragraph. Be specific. No more than 120 words.

Appendix C — The Eight Archetypes

Each archetype is a vector over ~25 features, weighted. Below are the high-level definitions with their dominant features. Full numeric weights live in `src/jobpilot/fit/archetypes.py` and are tunable.

The Founder Track

Who picks it: Fresh grads, aspiring founders, people who want range and pace.

Signals: Headcount small/mid (+), Series A-C funding (+), founder named on careers page (+), equity-heavy comp (+), "wear many hats" / "end-to-end ownership" language (+), travel low or moderate (neutral). Negative on enterprise size, defined leveling, multi-year tenure norms.

The Corporate Climber

Who picks it: Mid-career, ambition for title/scope, structure-oriented.

Signals: Headcount large/enterprise (+), public or late-stage (+), defined leveling (L4/L5, IC4) (+), MBA-heavy senior leadership (+), "high-performance" review cycles mentioned (+). Negative on "flat hierarchy", titles ambiguous, < 200 headcount.

The Stable Provider

Who picks it: Mid-career, family-focused, predictability over upside.

Signals: Parental leave ≥ 12 weeks (+), childcare or fertility benefits (+), healthcare strong (+), < 45-hour weeks per reviews (+), `wlb_score` ≥ 4.0 (+), tenure averages ≥ 3 years (+). Negative on "fast-paced", "hustle", recent layoffs.

The Women-Forward Workplace

Who picks it: Women prioritizing cultures with female leadership and lived equity.

Signals: $\geq 30\%$ women in named leadership (+), gender pay equity disclosure (+), mentorship programs mentioned (+), ERGs visible (+), positive sentiment from female-tagged Glassdoor reviews (+), parental leave parity (+). Negative on "bro-culture" mentions in reviews, all-male leadership pages, recent gender-discrimination news.

The Globe-Trotter

Who picks it: People who want travel as part of the job.

Signals: International offices ≥ 3 (+), "travel up to 30/50/75%" in JD (+), customer-facing or field roles (+), frequent-flier mentions in reviews (+). Negative on "fully remote, no travel", small one-office shops.

The Mission Believer

Who picks it: Work itself must matter — climate, health, education, equity.

Signals: Sector tags (climate, healthtech, edtech, civictch) (+), B-Corp or non-profit partnerships (+), mission concreteness with metrics (+), reviews mention impact positively (+). Negative on adtech, surveillance tech, undisclosed business model.

The Deep Specialist

Who picks it: Senior ICs who want technical depth and respect.

Signals: Staff/Principal/Distinguished engineer ladder visible (+), conference talks or peer-reviewed publications by employees (+), "no-management track required" stated (+), engineering blog active (+). Negative on "player-coach" requirement, tiny teams without seniority above your level.

The Remote-First Lifer

Who picks it: Async-friendly, distributed, no RTO whiplash.

Signals: "Remote-first" stated (not "remote-friendly") (+), distributed across ≥ 3 timezones (+), public RTO policy stable for 18+ months (+), async tools mentioned in reviews (+), no recent return-to-office news (+). Negative on hub policies, "hybrid 3 days a week", recent RTO mandates.

End of document